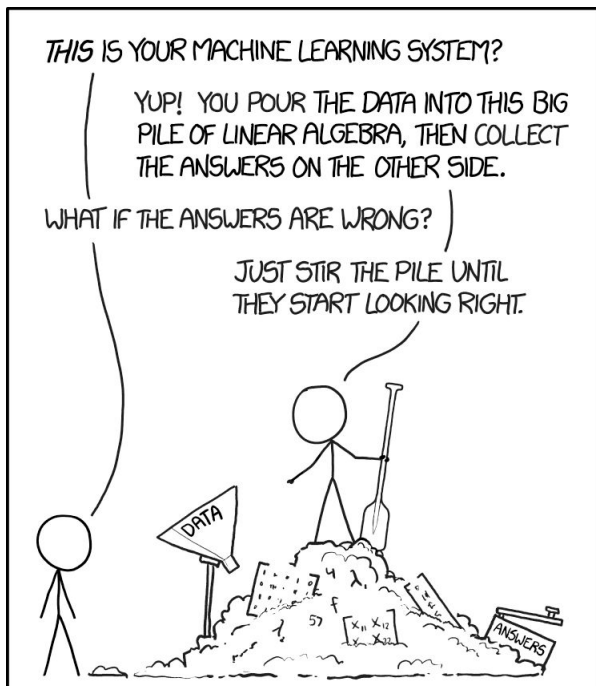


Evidently AI: ML Observability and Monitoring



Two Worlds of Software: Deterministic vs. Probabilistic

Errors aren't always $1+1=2$,; it's
learned from statistical patterns..

Silent Failure: ML vs. Code

- ML model fails silently with incorrect output
- Traditional software fails with obvious errors
- ML models can confidently give wrong answers
- Monitoring crucial for ML system reliability



Why is this different from other traditional software?

- Traditional Software: Code is deterministic. If $1 + 1 = 2$ today, it will tomorrow. Bugs are usually obvious (e.g., a crash).
- ML Systems: Are built on code + data. The code can be perfect, but if the data changes, the model's logic breaks.
- Failure is Silent: A model won't crash if it sees new data. It will just confidently give you the wrong answer. This is the core challenge of ML Observability

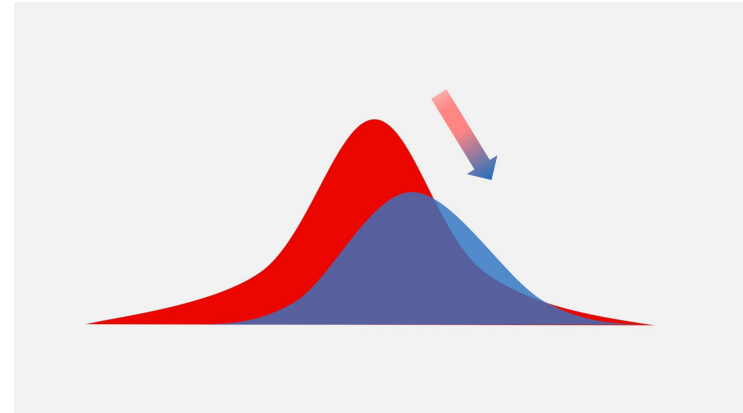
So what can go wrong?

Once deployed, a model's performance is not static. It *will* degrade.

- Training-Serving Skew: The data in production is immediately different from the training data (e.g., feature pipeline bug).
- Data Quality Issues: The "silent killers."
- Broken pipelines (e.g., a sensor fails, sending **NULLs**).
- Data schema changes (e.g., an upstream API changes "USA" to "United States").
- Data loss, corruption, or unexpected values.
- This is called **Data drift or concept drift... (more on this later)**

Data-drift: $P(X)$ (shift in distribution): Data Itself Changes

- Data drift is a shift in the distributions of the ML model input features.
- In simple terms, data drift is a change in the model inputs the model is not trained to handle. Detecting and addressing data drift is vital to maintaining ML model reliability in dynamic settings.



Example: Predicting demand for online sales..

Like 'store location

Incoming data



Feature distribution: sales_channel



Model quality: accuracy over time



TikTok slang and AI



- Social media shapes modern language
- New slang emerges from online trends
- Voice assistants adapt to evolving speech
- Language models require continuous updates

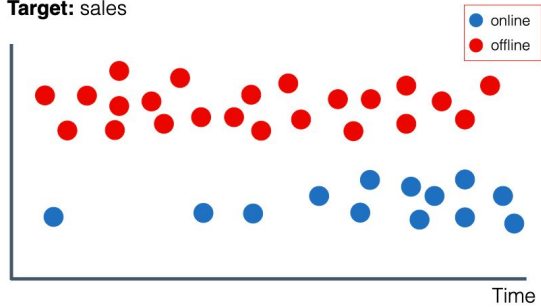
Data drift is not concept drift

- Data drift is a change in the input data. Concept drift is a change in input-output relationships. Both often happen simultaneously.
- While data drift describes changes in the data distribution, **concept drift** relates to changes in the relationships between input and target variables. Basically, concept drift means that whatever your model is predicting – it is changing.



Data drift

Target: sales

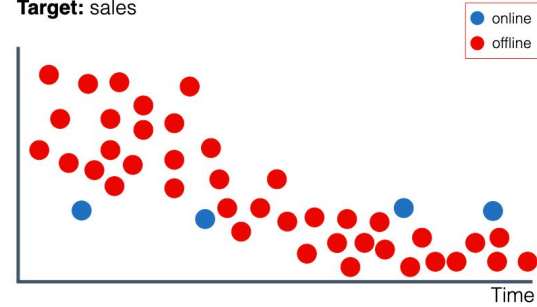


Feature distribution: sales channel



Concept drift

Target: sales



Feature distribution: sales channel



- Concept drift examples: a new competitor's discounts altering shopper behavior and decreasing average basket size, leading to inaccurate forecasts. COVID-19 also transformed shopping habits and disrupted logistics.
- **The difference:** Data drift refers to the shifts in input feature distributions, whereas concept drift refers to shifts in the relationships between model inputs and outputs.
- **The similarity:** Both data drift and concept drift can result in a decline in model quality and often coincide. In monitoring, data distribution drift can be a symptom of concept drift.

Prediction Drift: The Symptoms Appear ($P(Y)$)

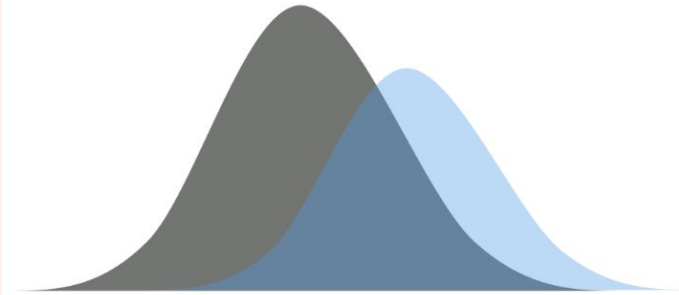
Prediction drift is the distribution shift in the model outputs





Input data

DRIFTED



Predictions

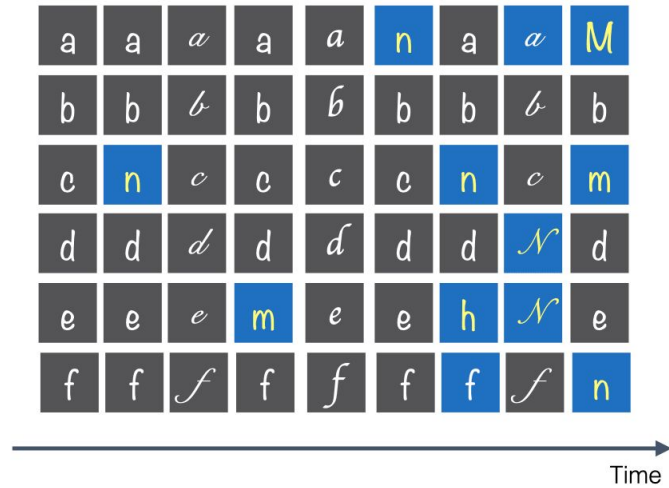
DRIFTED

- **The difference:** data drift refers to the changes in the model input data, while prediction drift refers to the changes in the model outputs.
- **The similarity:** both data and prediction drift are useful techniques for production model monitoring in the absence of ground truth and can signal the change in the model environment.

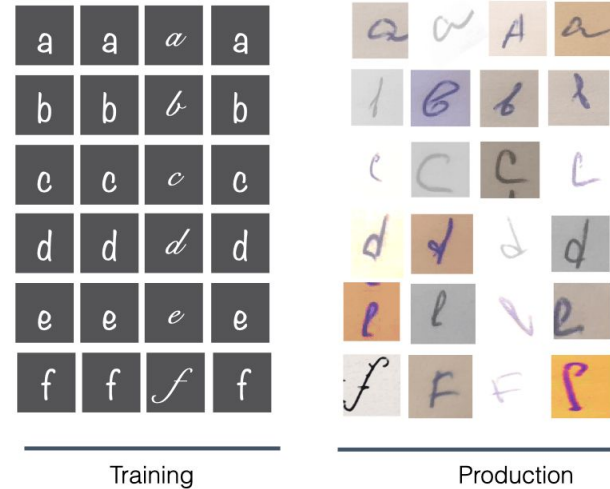
Train serve skew

- Training-serving skew is a mismatch between training and production data.
Data drift is a shift in the distribution of production data inputs over time.

Data drift



Train-serving skew

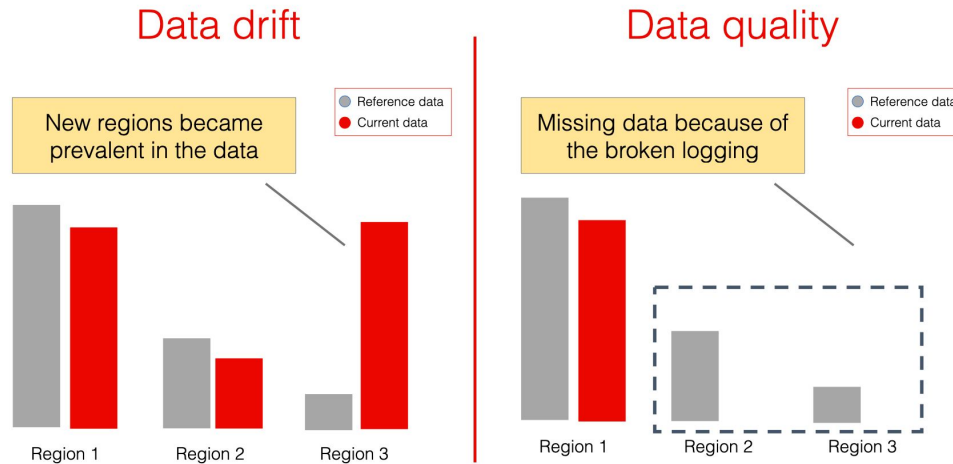


Why it occurs?

- A training-serving skew can occur if features available during training cannot be computed in production or are delayed, hindering model performance.
- This skew can also arise if the model is trained on synthetic or external datasets that don't fully match the application environment.
- The difference: data drift refers to gradual changes in input data distributions. Training-serving skew refers to a mismatch visible shortly after model production use, potentially including issues unrelated to environmental changes.
- The similarity: both refer to changes in input data. Similar distribution comparison techniques can detect input data drift and training-serving skew by contrasting production data with training.

Data Quality

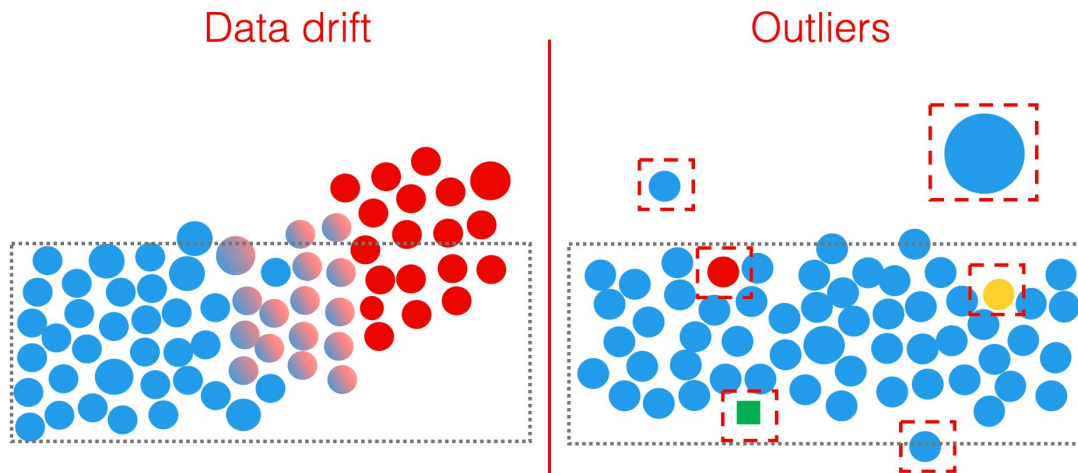
- Data quality issues refer to the data bugs, inconsistencies, and errors.



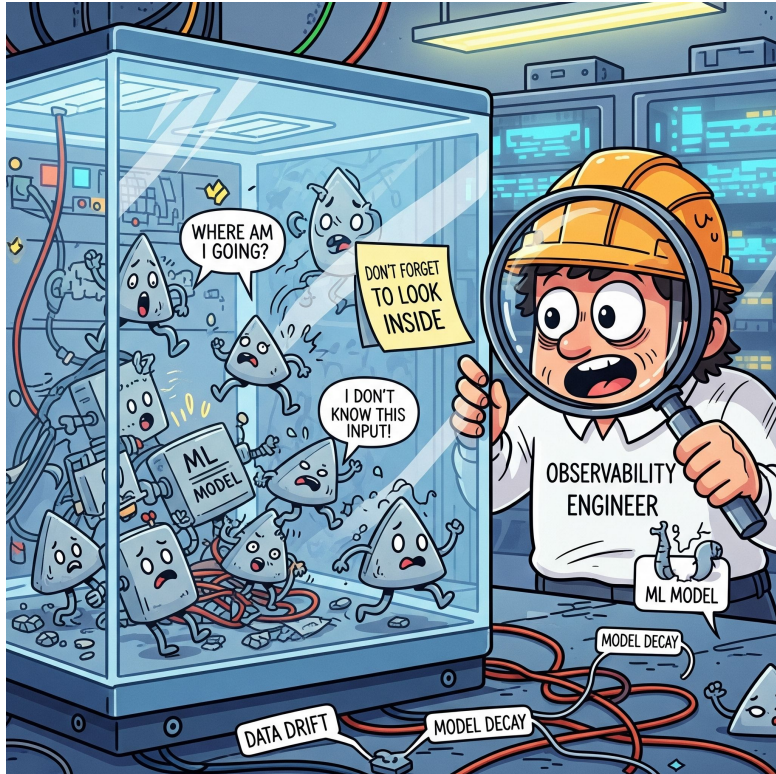
- An accidental change in feature scale can cause a statistical shift in distribution. It's helpful to categorize checks: first, verify data quality (completeness, relevant features non-negative), then apply data distribution checks for statistical shifts. This prevents needing to rule out data quality issues as a root cause of detected drift.
- The difference: Data quality addresses issues like missing values or errors. Data drift refers to statistical changes in data distribution, even with high-quality data. Data quality issues can cause observed data drift but are distinct.
- The similarity: Both data quality issues and data drift can degrade model quality and represent data changes. Drift detection techniques often reveal data quality issues.

Data Drift is NOT Outlier detection..

- **Drift detection** refers to the "global" data distributions in the whole dataset. **Outlier detection** serves a different purpose. You want to identify individual objects in the data that look different from others.



Machine Learning models are not “set it and forget it” ...



Why is this important?

1. It's the **Earliest Warning System**: Model quality metrics (like accuracy) often require ground truth labels, which can be delayed by days or weeks (e.g., waiting to see if a loan *actually* defaults). Data drift can be measured immediately.
2. It's the **Root Cause of Model Failure**: Monitoring data drift helps you debug *why* your model's accuracy is dropping.
3. It **Drives Model Maintenance**: Detecting drift is the primary trigger for deciding when to retrain, update, or rebuild your model.
"Set it and forget it" does not work for ML.

Solutions..

How do you "measure" if two datasets are different? You compare a "reference" dataset (e.g., training data) to a "current" dataset (e.g., last 24 hours).

- Summary Statistics:
- Easiest method. Compare **mean**, **median**, **variance**, **min/max**, **quantile** values.
- Good for a quick check but can miss complex changes.
- Statistical Tests:
- Use hypothesis testing to see if the two samples come from the same distribution.
- Examples: Kolmogorov-Smirnov (KS) test, Chi-Squared test.
- Output: A p-value. If **$p < 0.05$** , you "detect drift."
- Distance Metrics:

Quantify how far apart* the two distributions are.

- Examples: Population Stability Index (PSI), Wasserstein (Earth-Mover) Distance, Jensen-Shannon Divergence.
- Output: A "drift score." This is often more interpretable than a p-value.

The "Large Dataset" Problem

A major challenge in production: Statistical tests are often *too sensitive* on large datasets.

With millions of data points, even a tiny, practically meaningless shift in the data (e.g., mean age changes from 40.1 to 40.2) will return a "statistically significant" p-value.*

- This leads to "alert fatigue"—you get p-value alerts every day and start ignoring them.

Key Insight: For large datasets, you don't care about statistical significance. You care about practical significance or the magnitude of the drift.*

Better Metrics for Large Datasets

This is why Distance Metrics are often preferred in production monitoring systems:

- Wasserstein Distance (Earth-Mover Distance):
- Measures the "cost" of turning one distribution into the other.
- It's less sensitive to small, insignificant changes and gives a score (e.g., "0.2") that quantifies the drift magnitude.
- Population Stability Index (PSI):
- Very popular in finance. Bins the data and compares the percentage of data in each bin.
- Rule of thumb:
- $PSI < 0.1$: No significant change.
- $0.1 < PSI < 0.25$: Minor drift.
- $PSI > 0.25$: Major drift.

What About Text Data? (NLP Models)

You can't just run a KS-test on a `review_text` column. So how do you monitor NLP models?

Approach 1: Text Descriptors

Don't monitor the text itself; monitor statistics about the text.*

- Examples:
- Text length (number of characters or words).
- Share of out-of-vocabulary (OOV) words.
- Share of non-letter symbols (e.g., punctuation, HTML tags, emojis).
- Presence of different languages.
- This is a great way to catch data quality issues (like HTML tags leaking into your data).
-

But what to do??

- Retrain your models... obviously
- Feedback delay is a time lag between model predictions and receiving feedback on those predictions. Monitoring input data distribution drift is a valuable proxy when ground truth labels are unavailable.
- Summary statistics
- Statistical tests:
 - **Kolmogorov Smirnov** for numerical features or the **Chi-square test**
 - The output (and a “drift score”) of a statistical test is a **p-value** that serves as a “measure of confidence.”

Rule checks...

Lastly, you can perform some drift checks using straightforward rules. For example, you might choose to be alerted whenever:

The share of the predicted “fraud” category is over 10%.

A new categorical value appears in a feature “location” or “product type.”

More than 10% of the feature “salary” values are out of the defined min-max range.

What is Evidently AI?

- An open-source Python library specifically designed for ML observability, monitoring, and debugging.
- It implements all the concepts we just discussed (data drift, concept drift, data quality) right out of the box.
- It generates interactive Reports (for debugging) and live Dashboards (for monitoring).

How Evidently Solves These Problems

Evidently is built to handle real-world production challenges:

Generates Comprehensive Reports: You don't just get a single number. You get interactive plots for every feature, showing you exactly how it drifted.*

- Smart Defaults: It automatically chooses the right drift detection method based on your feature type and data size (e.g., KS-test for small numerical data, Wasserstein for larger data, Chi-Squared for categorical).
- Built-in Text Monitoring: Implements `TextDescriptorsDriftMetric` and the "domain classifier" approach (`ColumnDriftMetric` on text) automatically.

Tracks Quality: If you do have labels, it can track accuracy, F1, precision, recall, and plot a confusion matrix to show you where your model is failing.*

"My Data Drifted. What's Next?"

Detecting drift is just the first step. Here is the typical "response plan":

1. Analyze & Diagnose:

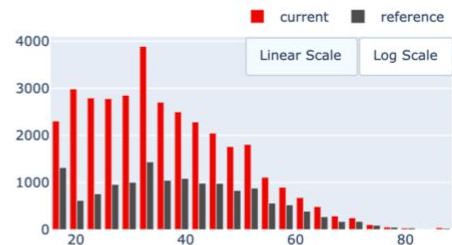
- Use a tool like Evidently to understand the drift.
- Is it a data quality bug? (e.g., HTML tags). If yes, fix the pipeline. Do not retrain on bad data.
- Is it a real-world change? (e.g., new user segment). If yes, proceed to step 2.
- Is it a false positive? (e.g., KS-test on large data). If yes, adjust your monitoring sensitivity (e.g., switch to Wasserstein distance).

1. Retrain (if needed):

- If it's a real, permanent change, you need to update your model

age
num

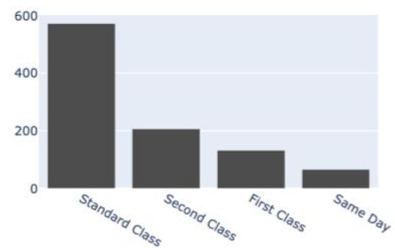
	current	reference
count	34687	14155
mean	37.97	40.3
std	13.31	14.52
min	17.0	17.0
25%	27.0	29.0
50%	36.0	39.0
75%	47.0	50.0
max	90.0	90.0
unique	73 (0.21%)	73 (0.52%)
most common	23.0 (3.12%)	17.0 (4.07%)
missing	0 (0.0%)	0 (0.0%)
infinite	0 (0.0%)	0 (0.0%)



Ship_Mode

cat

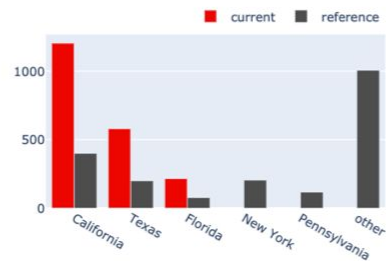
	current	reference
count	0	976
unique	0 (0.0%)	4 (0.2%)
most common	nan (100.0%)	nan (51.2%)
missing	2000 (100.0%)	1024 (51.2%)
new categories	0	
missing categories	4	



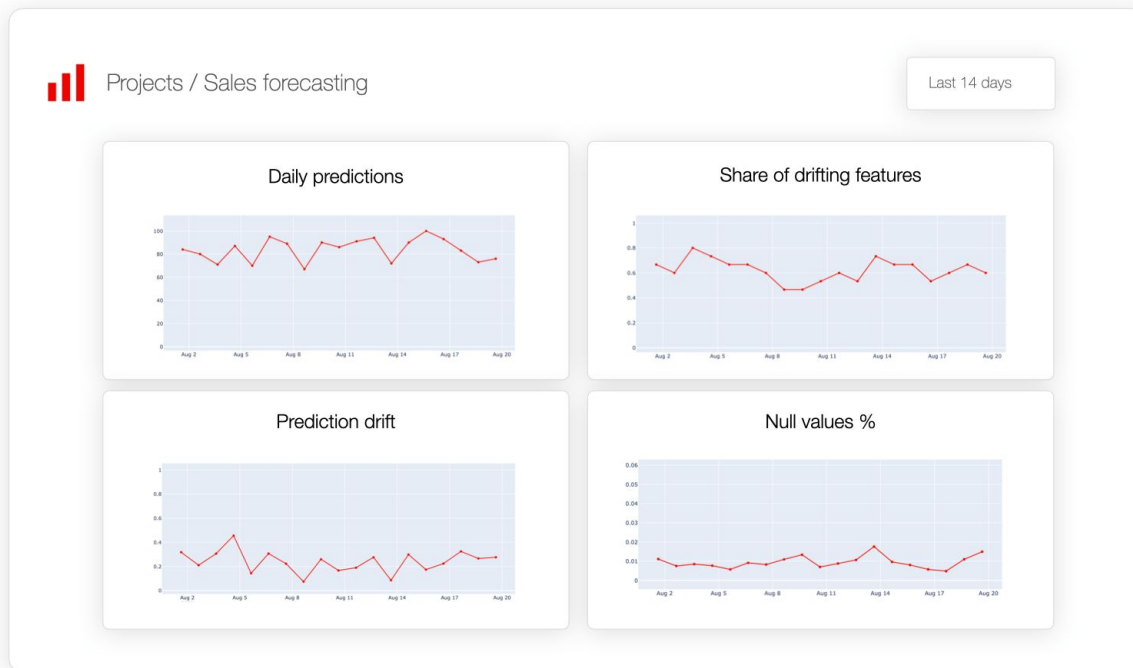
State

cat

	current	reference
count	2000	2000
unique	3 (0.15%)	48 (2.4%)
most common	California (60.2%)	California (19.95%)
missing	0 (0.0%)	0 (0.0%)
new categories	0	
missing categories	45	



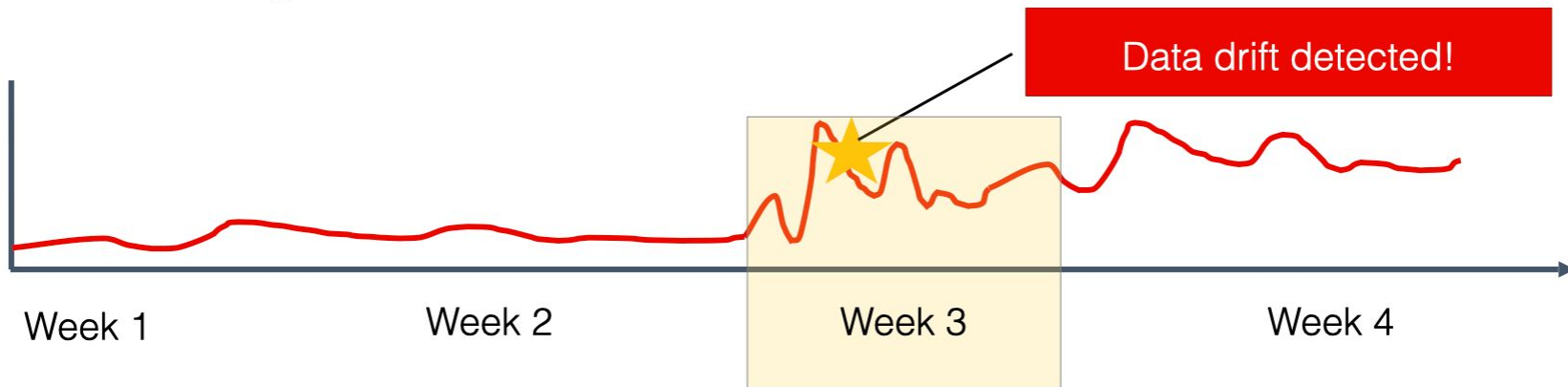
Typically something like this...



Accuracy over time

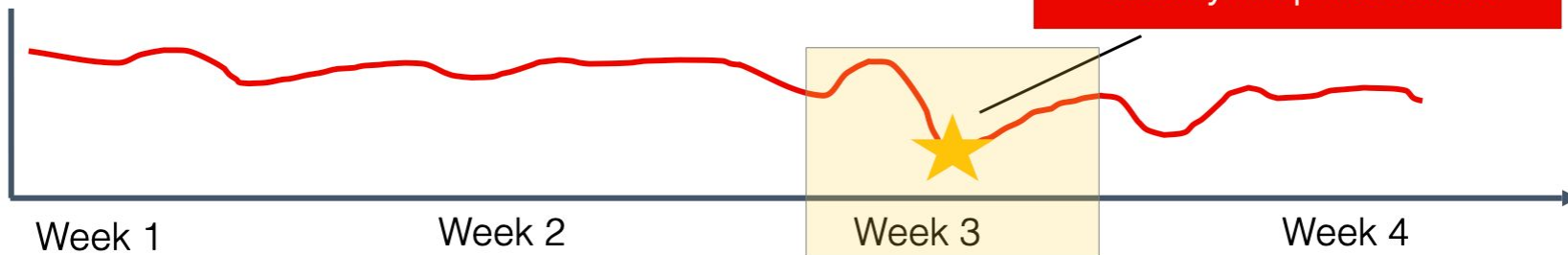


Share of **drifting** features



Accuracy over time

Quality drop detected!



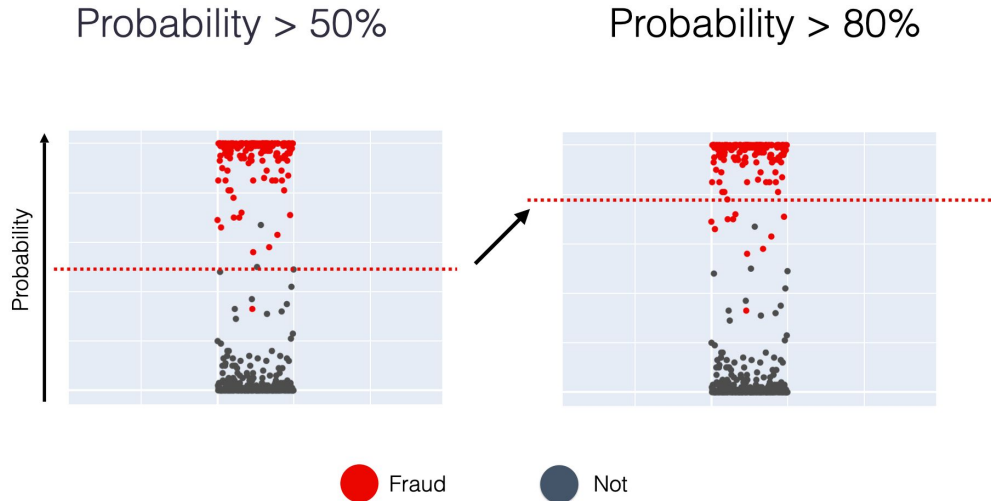
Model drift alert!

10 features out of 30 are drifting.
Here is a summary...

	Feature	Type	Reference Distribution	Current Distribution	Data drift	Impact on Accuracy
1	user_created_posts	int			Detected	0.00002
2	user_created_posts	int			Detected	0.00001
3	user_created_posts	int			Detected	0.00009
4	user_created_posts	int			Detected	0.00001
5	user_created_posts	int			Detected	0.00001

What if retraining is not an option?

- You can also consider **modifying the decision process** on top of the model output



Model Redesign

- Go back to the drawing board..
- Last resort
- A frequent approach is feature selection. Before utilizing some of the available features in model training, you can review their historical variability and filter out the features with significant historical drifts. You can also consider applying some feature engineering techniques, such as bucketing volatile numerical features into a limited number of categories.
- Sometimes, it is worthwhile to choose a model that is less performant on a historical evaluation set but is more robust to data shifts. You can also consider using domain-based or causal models at the core of your system, which can be more reliable in case of data changes.
-